Heston Model Calibration with Gaussian Process & Carr-Madan Fast Fourier Transform

Ayush Dhital, Michael Gabe, Chloe Zheng

July 2026

Contents

1 Introduction

The Black–Scholes-Merton (BSM) model is a foundational option-pricing framework, but its assumption of constant volatility limits its ability to reproduce features observed in real markets, such as the volatility smile. This project therefore considers the Heston model, in which the asset price and its variance follow coupled stochastic processes, allowing it to capture the skew that BSM structurally cannot. We use the Carr-Madan FFT method as our primary pricing benchmark: it prices the Heston model semi-analytically rather than through simulation, giving an accurate and fast reference.

Calibrating the Heston model against real market data is computationally expensive: each iteration of a gradient-based optimizer requires repeatedly re-evaluating the pricer across the full option chain. Therefore, we develop Gaussian Process (GP) and sparse Gaussian Process (SGP) emulators that learn the mapping from Heston parameters and option characteristics to implied volatility offline (on synthetically generated data), so that calibration can query the emulator rather than the pricer directly. This approach follows Mongwe et al. (2025) [4], who demonstrate that GP-based calibration can match the accuracy of standard techniques while additionally producing calibrated uncertainty estimates at each point on the volatility surface - a property neither FFT nor Monte Carlo pricing provides natively. Motivated by this, we evaluate GP and sparse GP emulators along two axes suggested by the textbook's framework: computational speed relative to direct FFT and Monte Carlo calibration, and the extent to which their predictive uncertainty correctly identifies regions of the SPY options chain where calibration is less reliable, such as short-dated, thinly-quoted contracts.

As shown in Figure 1, the project workflow consists of generating synthetic Heston data, training and validating the emulators using an 80/20 data split, preparing a real SPY options dataset, calibrating the model using GP, sparse GP, FFT, and Monte Carlo methods, and comparing their accuracy, computational efficiency, and predictive uncertainty.

2 Heston Model

This section documents the analysis conducted in the notebook `heston_parameter_review_extremes.ipynb`.

We introduce the Heston stochastic volatility model, explain its relevance to the project, and examine the roles of its key parameters. We then use numerical simulations to validate our implementation and illustrate how changes in the model parameters affect the resulting stock-price and variance paths.

2.1 Model Specification

The Heston model [3] is a stochastic volatility model in which the asset price S_t and its instantaneous variance v_t evolve jointly according to the coupled system of stochastic differential equations

$$dS_t = \mu S_t\,dt + \sqrt{v_t}\,S_t\,dW_t^S, \tag{1}$$

$$dv_t = \kappa(\theta - v_t)\,dt + \sigma\sqrt{v_t}\,dW_t^v, \tag{2}$$

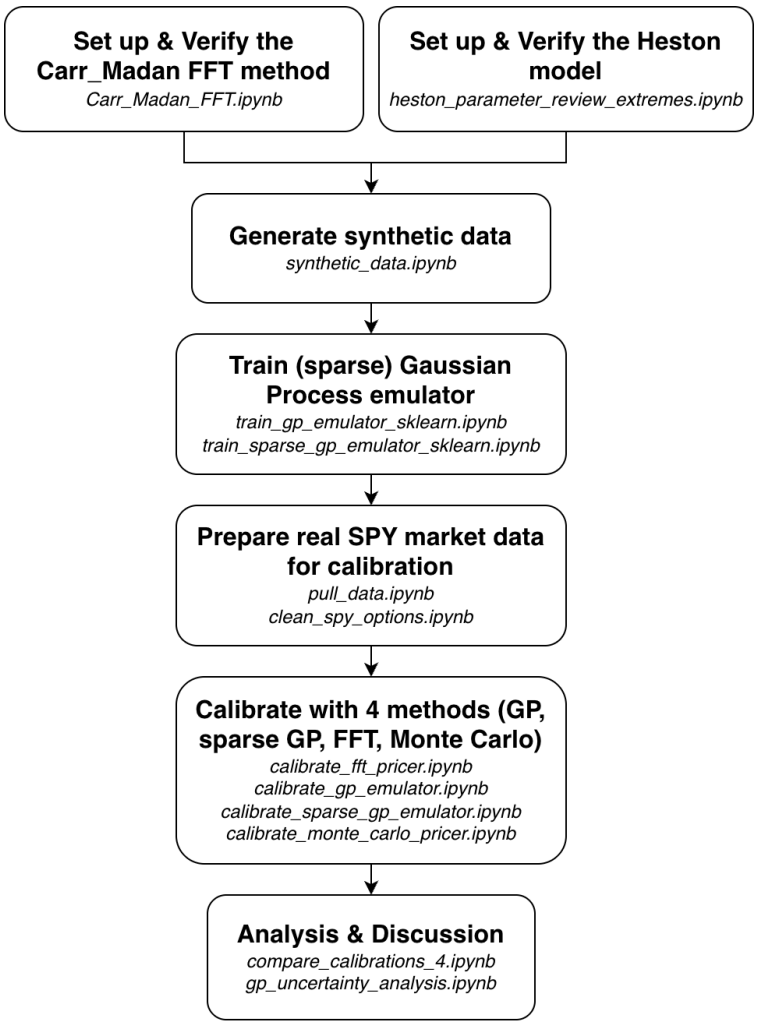

Figure 1: Workflow of the project

with the two driving Brownian motions correlated according to

$$dW_t^S \, dW_t^v = \rho \, dt. \tag{3}$$

Unlike the Black-Scholes-Merton model, in which volatility is a fixed constant, the Heston model allows volatility to evolve as its own random process, coupled to the asset price. The Heston model has 5 parameters that describe how a stock's volatility behaves over time:

- $\boldsymbol{v_0}$ - the initial variance, setting the starting level of volatility;
- $\boldsymbol{\theta}$ - the long-run variance, the level toward which v_t reverts;
- $\boldsymbol{\kappa}$ - the mean-reversion speed;
- $\boldsymbol{\sigma}$ - the volatility of variance;
- $\boldsymbol{\rho}$ - the correlation between asset-price and variance innovations.

The calibration process we carry out aims at finding the best fitted values for these parameters.

A subtlety of this model is that the variance v_t remains strictly positive for all t, only when the parameters satisfy the *Feller condition* [2]

$$2\kappa\theta \geq \sigma^2. \tag{4}$$

When this condition is violated, v_t can reach zero, though the mean-reverting drift term prevents it from being absorbed there. In all of our subsequent analysis, we ensure that this condition is never violated by choosing values of κ, θ, and σ that satisfy Eq. (4).

2.2 Stock Price and Variance Paths in the Heston Model

Given a set of initial parameter values, the Heston model can be used to simulate both stock price and variance paths. In this section, we examine how changes in each parameter affect the simulated stock price and variance processes and discuss the financial interpretation of each parameter. Unless otherwise stated, the baseline parameter values used throughout the following subsections are given in Table 1.

Parameter	**Value**	**Description**
S_0	100	Initial stock price
v_0	0.08	Initial variance
θ	0.04	Long-run mean variance
κ	2.5	Mean-reversion rate
σ	0.06	Volatility of variance (vol-of-vol)
ρ	-0.7	Stock–variance correlation
μ	0	Drift term
N	504	Number of time steps over two years

Table 1: Baseline parameter values used in the Heston model simulations.

The initial stock price is set to $S_0 = 100$, while the initial variance is $v_0 = 0.08$. The long-run mean variance is given by $\theta = 0.04$, and the mean-reversion rate is set to $\kappa = 2.5$. The volatility-of-volatility parameter is $\sigma = 0.06$, while the correlation between the stock price and variance processes is $\rho = -0.7$. The drift term is taken to be $\mu = 0$.

The simulations are performed over a two-year period using $N = 504$ time steps, corresponding to approximately 252 trading days per year. Therefore, the time increment is

$$\Delta t = \frac{2}{504} = \frac{1}{252}.$$

To study the effect of each parameter, one parameter is varied at a time while all other parameters are held fixed at the baseline values shown in Table 1. This allows the influence of each parameter on the stock price and variance paths to be examined separately. ‘‘‘

2.2.1 Mean-Reversion Rate (κ)

The parameter κ controls how quickly the instantaneous variance v_t reverts toward its long-run level θ, on average. This is the Heston model's analogue of the constant-volatility assumption in the BSM model: as $\kappa \to \infty$ (or equivalently, as the vol-of-vol parameter $\sigma \to 0$), v_t is pinned arbitrarily close to θ for all t, and the model collapses toward BSM-like dynamics with a single, effectively constant volatility level.

Figures 2 and 3 compare simulated paths for $\kappa = 0.25$ and $\kappa = 8$, holding all other parameters fixed. With a small mean-reversion rate, the latent variance can remain far from θ for extended periods, since the pull back toward the long-run level is weak. With a larger mean-reversion rate, v_t is pulled back toward θ much more quickly and crosses it far more frequently, staying tightly clustered around the long-run level throughout the simulation.

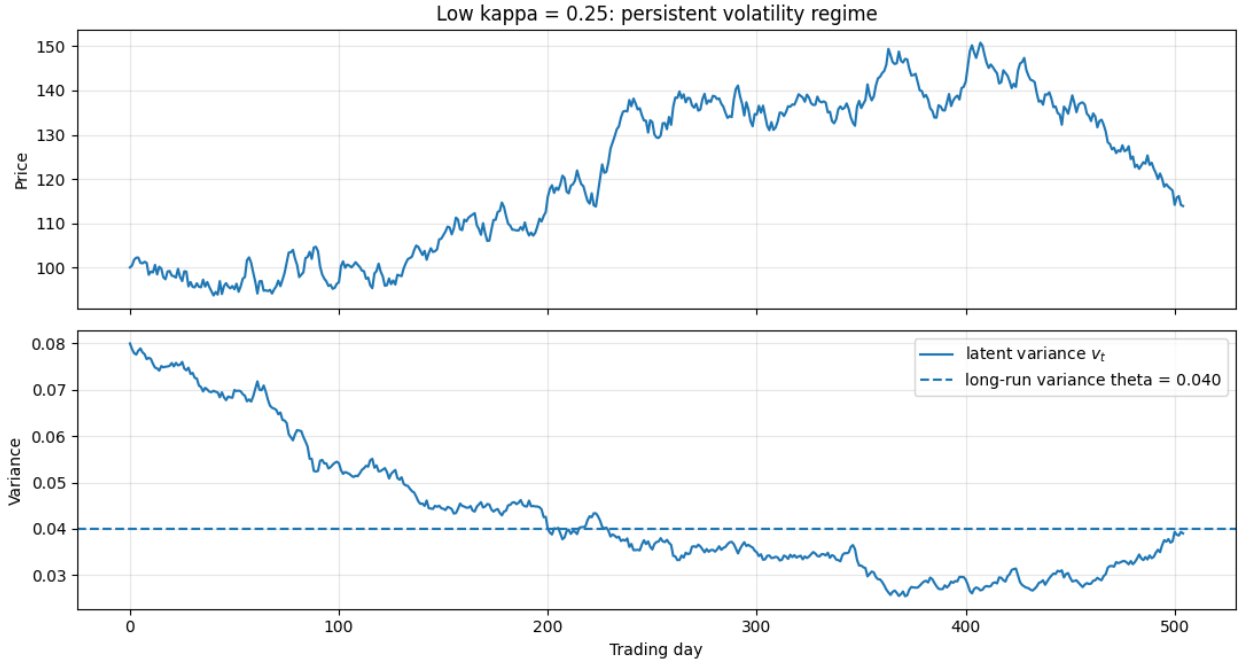

Figure 2: Stock price and latent variance paths for $\kappa = 0.25$. With slow mean reversion, v_t drifts away from θ for long stretches of time.

2.2.2 Long-Run Variance (θ)

The parameter θ sets the long-run volatility, towards which the instantaneous variance v_t reverts, and can be interpreted as the average variance the asset is expected to exhibit over long horizons. Since v_t continually reverts toward θ (at rate κ), increasing θ raises the level around which the latent variance fluctuates, without necessarily changing how far v_t wanders from that level at any given time. The spread is governed instead by κ and the vol-of-vol parameter σ. A higher θ correspondingly widens the typical range of stock-price fluctuations, since the asset spends more time exposed to higher variance and the resulting price path exhibits larger swings on average.

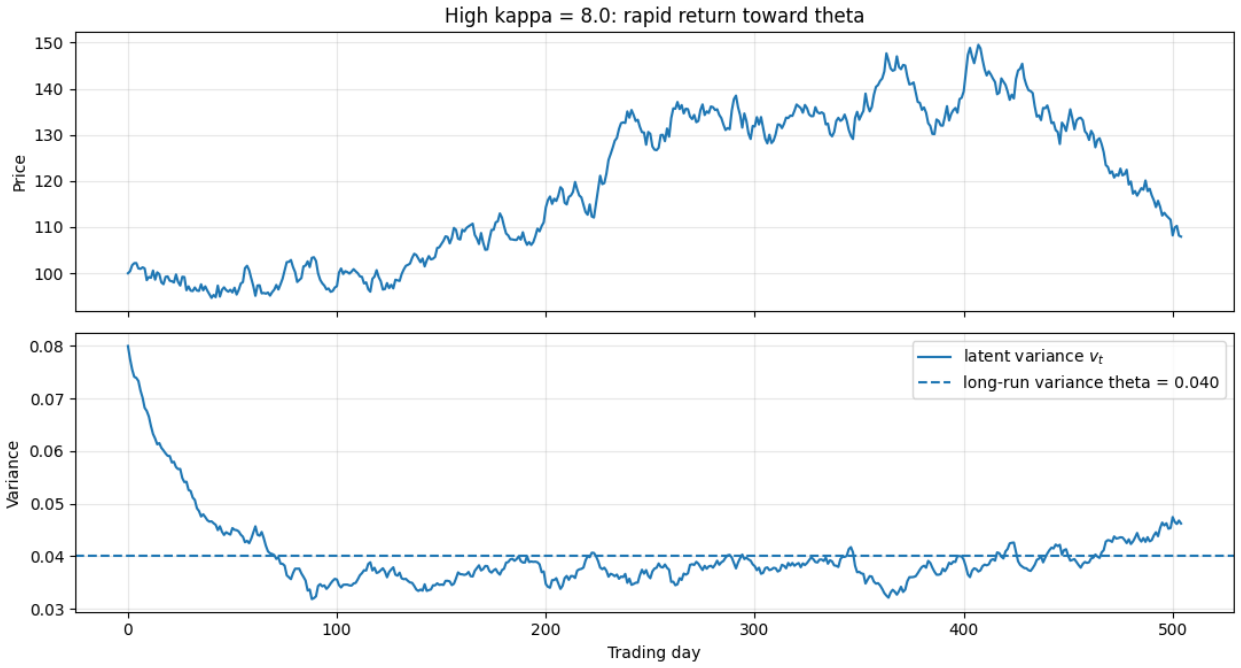

Figure 3: Stock price and latent variance paths for $\kappa = 8$. With fast mean reversion, v_t crosses θ far more often and remains close to it throughout.

Figures 4 and 5 compare simulated paths for $\theta = 0.01$ (10% long-run volatility) and $\theta = 0.16$ (40% long-run volatility), holding all other parameters fixed. Since v_0 differs from θ in both cases, the latent variance can be seen adjusting away from its starting value and toward the respective long-run level early in each path, before settling into fluctuations around θ. Also, it is clear from Figures 4 and 5 that a higher-θ path exhibits larger stock-price swings.

2.2.3 Stock-Volatility correlation (ρ)

The parameter ρ governs the instantaneous correlation between the stock price and its variance. When ρ is strongly negative, downward moves in the stock price tend to coincide with upward jumps in variance. When $\rho = 0$, stock price and variance are independent. When ρ is strongly positive, the effect reverses: rising prices tend to coincide with rising variance.

Figures 6-8 compare simulated paths for $\rho = 0.95$, $\rho = 0$, and $\rho = -0.95$, holding all other parameters fixed. The variance paths themselves look similar across the three cases, since ρ does not affect the dynamics of v_t in isolation; the difference instead shows up in how the stock price co-moves with the variance.

3 Carr-Madan FFT Setup for Heston Model

This section documents the analysis conducted in the notebook `Carr_Madan_FFT.ipynb`.

The Black-Scholes-Merton (BSM) model is widely used to price options for a given strike price and time to maturity under the assumption of constant volatility. The Heston model

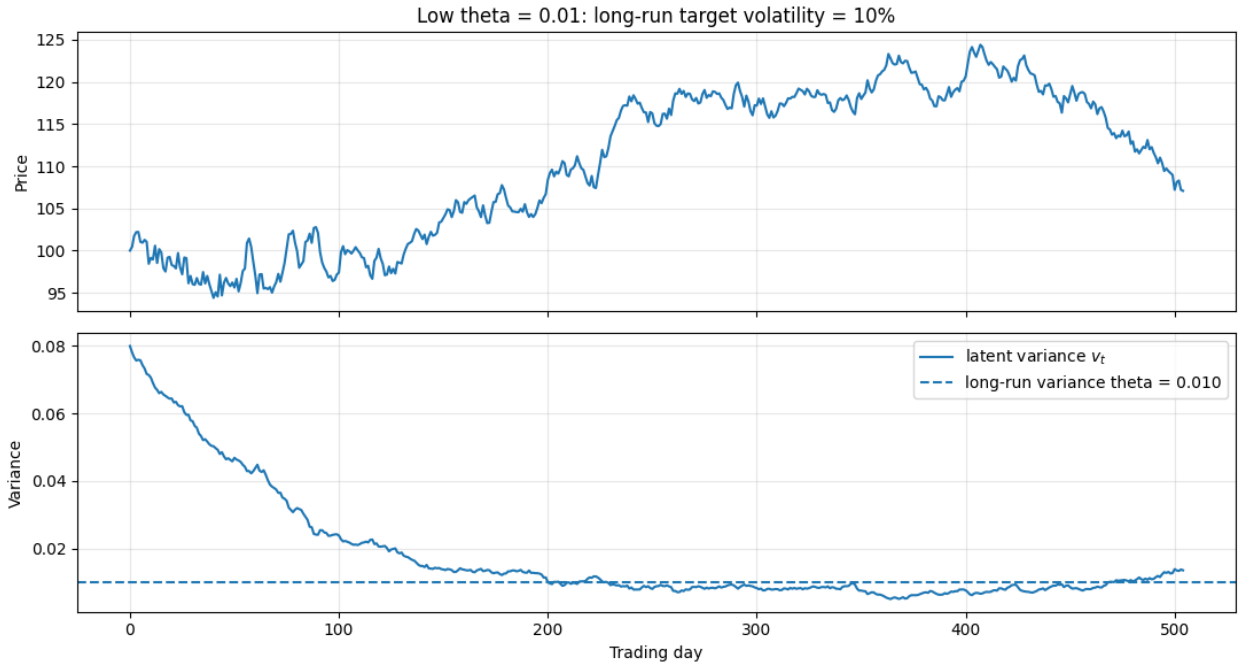

Figure 4: Stock price and latent variance paths for $\theta = 0.01$ (10% long-run volatility), with $v_0 = 0.08$. Variance drifts downward from its initial level toward the lower long-run target.

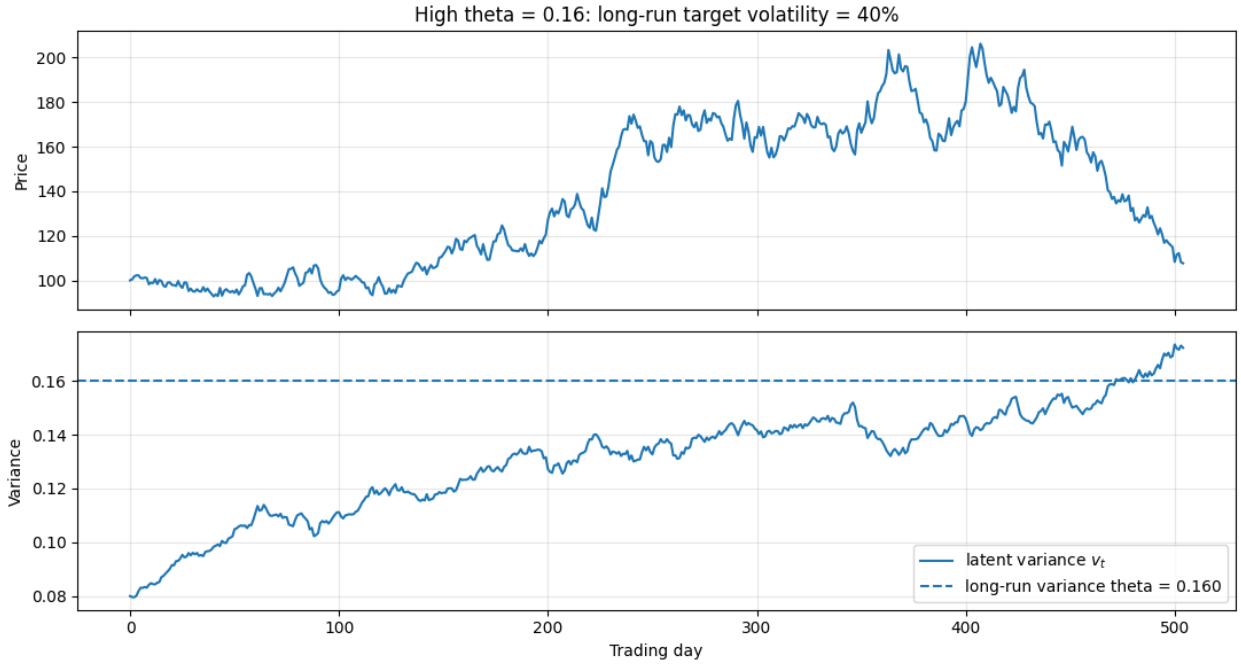

Figure 5: Stock price and latent variance paths for $\theta = 0.16$ (40% long-run volatility), with $v_0 = 0.08$. Variance drifts upward from its initial level toward the higher long-run target, producing larger stock-price swings.

Figure 6: Stock price and latent variance paths for $\rho = 0.95$. Rising variance tends to coincide with rising stock prices.

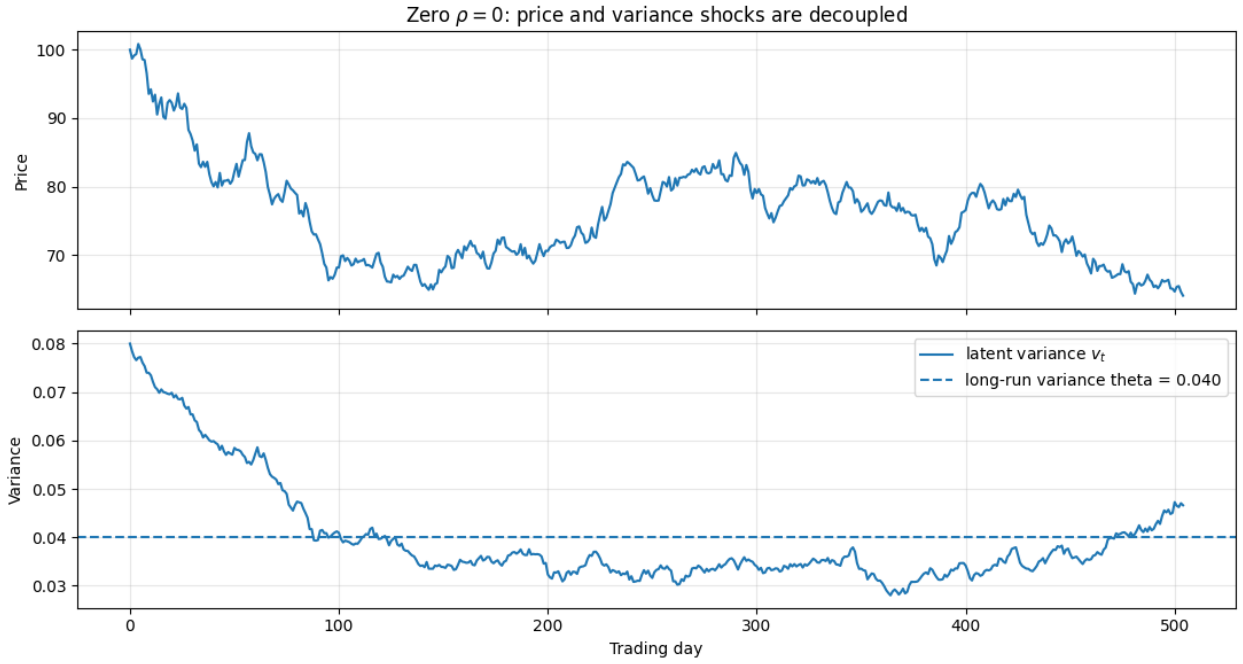

Figure 7: Stock price and latent variance paths for $\rho = 0$. Stock price and variance are independent, with no correlation between them.

relaxes this assumption by allowing volatility to follow a stochastic process, thereby capturing observed market features such as volatility smiles and skews. We then review the Carr-Madan

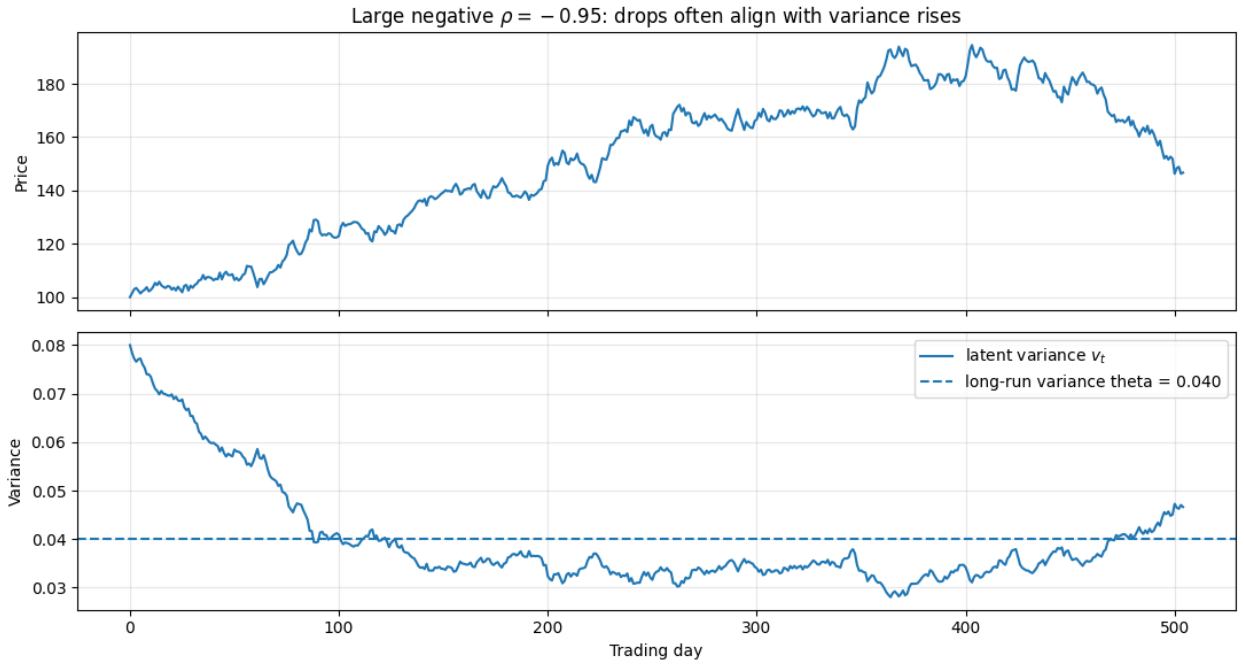

Figure 8: Stock price and latent variance paths for $\rho = -0.95$. Spikes in variance tend to coincide with drops in the stock price.

Fast Fourier Transform (FFT) method [1] for pricing European options from a model's characteristic function and specialize the method to the Heston model.

3.1 Setup: Modified Call Price and Its Fourier Transform

Let $k = \ln K$ denote the log-strike price, and let $C_T(k)$ denote the price of a T-maturity European call struck at e^k. Writing $s_T = \ln S_T$ for the log stock price at maturity and $q_T(s)$ for its risk-neutral density, the characteristic function of s_T is defined as

$$\phi_T(u) \equiv E^Q\left[e^{ius_T}\right] = \int_{-\infty}^{\infty} e^{ius}\, q_T(s)\, ds, \tag{5}$$

and the call price can be written directly as a discounted expectation of the payoff,

$$C_T(k) = e^{-rT} \int_k^{\infty} (e^s - e^k)\, q_T(s)\, ds. \tag{6}$$

As $k \to -\infty$, $C_T(k) \to S_0$, a nonzero constant; consequently $C_T(k)$ is not square-integrable in k over the whole real line, and its Fourier transform does not exist in the usual sense. Carr and Madan's resolution [1] is to instead work with a *damped* call price,

$$c_T(k) \equiv e^{\alpha k}\, C_T(k), \qquad \alpha > 0, \tag{7}$$

whose exponential decay as $k \to -\infty$ restores square-integrability, and to Fourier transform this modified quantity instead:

$$\psi_T(v) = \int_{-\infty}^{\infty} e^{ivk}\, c_T(k)\, dk. \tag{8}$$

Substituting the definitions (6) and (7) into (8) and swapping the order of integration gives

$$\psi_T(v) = \int_{-\infty}^{\infty} e^{ivk} e^{\alpha k} e^{-rT} \int_{k}^{\infty} (e^s - e^k)\, q_T(s)\, ds\, dk \tag{9}$$

$$= e^{-rT} \int_{-\infty}^{\infty} q_T(s) \int_{-\infty}^{s} \left(e^{s+\alpha k} - e^{(1+\alpha)k}\right) e^{ivk}\, dk\, ds. \tag{10}$$

The inner integral over k is elementary, yielding terms of the form $e^{(\alpha+1+iv)s}/(\alpha+iv)$ and $e^{(\alpha+1+iv)s}/(\alpha+1+iv)$; combining them and recognizing the remaining integral over s as ϕ_T evaluated at a shifted argument yields the closed form

$$\psi_T(v) = \frac{e^{-rT}\,\phi_T\big(v-(\alpha+1)i\big)}{\alpha^2+\alpha-v^2+i(2\alpha+1)v}. \tag{11}$$

This is the central result of the Carr-Madan method: it expresses the transform of the option price entirely in terms of the characteristic function ϕ_T of the log-price. Since $C_T(k)$ is real, $\psi_T(v)$ is Hermitian (even real part, odd imaginary part), so the inverse Fourier transform can be folded onto the positive half-line, recovering the call price as

$$C_T(k) = \frac{e^{-\alpha k}}{\pi} \int_0^{\infty} e^{-ivk}\, \psi_T(v)\, dv. \tag{12}$$

3.2 The Heston Characteristic Function

Equation (12) holds for any model with a known characteristic function; specializing to Heston, $\phi_T(u)$ is given in closed form by [3]

$$\phi_T(u) = \exp\big(iu \ln S_0 + C(u,T) + D(u,T)\, v_0\big), \tag{13}$$

where

$$D(u,T) = \frac{\kappa - \rho\sigma i u - d}{\sigma^2} \cdot \frac{1-e^{-dT}}{1-g\,e^{-dT}}, \tag{14}$$

$$C(u,T) = iuT\,r + \frac{\kappa\theta}{\sigma^2}\left[(\kappa-\rho\sigma i u - d)T - 2\ln\left(\frac{1-g\,e^{-dT}}{1-g}\right)\right], \tag{15}$$

with

$$d = \sqrt{(\rho\sigma i u - \kappa)^2 + \sigma^2(iu+u^2)}, \qquad g = \frac{\kappa - \rho\sigma i u - d}{\kappa - \rho\sigma i u + d}. \tag{16}$$

Here $\kappa, \theta, \sigma, \rho, v_0$ are the Heston parameters introduced in Section 2, and r is the risk-free rate. We take the underlying stock to be non-dividend-paying, consistent with the stock dynamics in (1); a dividend yield q can be incorporated by replacing r with $r-q$ throughout if needed. Substituting (13) into (12) recovers the Heston call price as a function of strike and maturity.

3.3 Validation: Heston-FFT vs BSM/Monte-Carlo comparison

Heston model is a generalized version of BSM model. If the parameter σ (vol-of-vol) is set to zero, and θ is set to initial volatility, then the Heston model reduces to BSM model. Here, we compare the price of a European call option between a BSM model and Heston model using Carr-Madan FFT method setting $\sigma = 0$ and $\theta = v_0$ for different strike prices (all maturity time is set to 1 year). We have also simulated 120,000 realizations of stock path using Heston model and computed the expected price of call option at maturity (which we refer to as Monte Carlo result). We compare the Monte Carlo result with the Carr-Madan FFT method and compute the error. We verify that our FFT computation is reliable and the absolute difference in call option value at maturity between MC and FFT is 0.0286. This is shown in Figure 9 below.

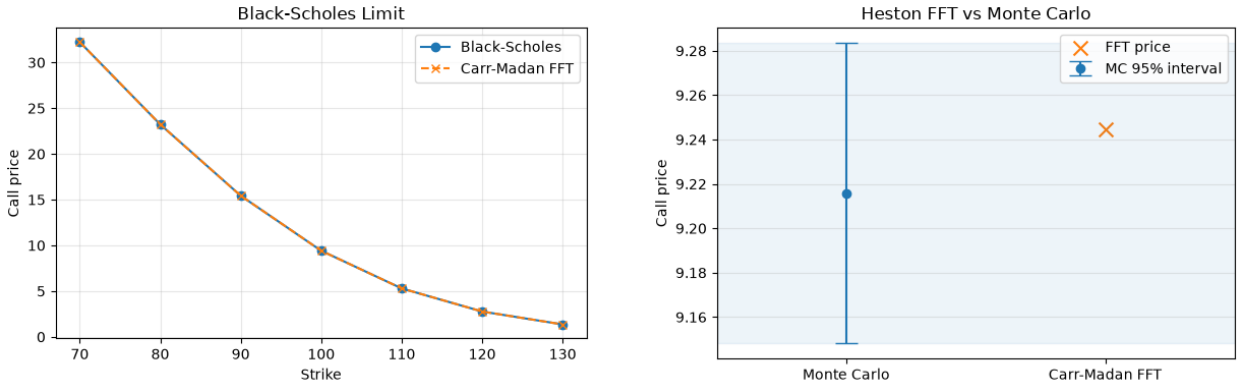

Figure 9: Comparing the price of a European Call option between Black-Scholes and Heston by setting the Heston parameters to yield the BS model. The price is computed using the Carr-Madan FFT in Heston model. We also compare this with Monte Carlo simulation (right) and the results from different methods all agree within uncertainty, thereby validating our FFT model.

4 Synthetic Data

This section documents the analysis conducted in the notebook `synthetic_data.ipynb`.

Before the Heston model can be calibrated to real market data, the GP and sparse GP emulators must first be trained, and training requires a labeled dataset of Heston parameters and their corresponding implied volatilities. We generate one such dataset synthetically: we sample a large number of plausible Heston parameter sets, price European options under each set using the Carr-Madan FFT pricer from Section 3, and convert the resulting prices into implied volatilities. Section 4.1 describes how the parameter sets are sampled, and Section 4.2 describes how each sampled set is turned into a synthetic implied-volatility surface.

4.1 Sampling

Plain uniform random sampling has a well-known weakness in higher dimensions: even though each point is individually random, pure chance means some regions of the space end up clustered with many nearby points while other regions receive almost none. This problem worsens as the number of dimensions increases, and is part of what is often called the "curse of dimensionality."

We therefore considers the Latin Hypercube Sampling (LHS), and utilize the function `scipy.stats.qmc.LatinHypercube` for implementation. LHS addresses this by dividing each parameter's range into N equal-probability bins, where N is the number of samples, and guaranteeing exactly one sample per bin per dimension. This forces even coverage along every axis individually, so the design cannot, for example, accidentally leave the low-κ range unsampled. LHS is not fully random, but it is far more evenly spread than uniform random sampling for the same number of samples.

The key distinction is that uniform random sampling does not optimize for coverage; it draws points independently and relies on averaging out over many samples. Space-filling designs such as LHS instead deliberately construct the sample set to cover the space evenly. This property is well suited to our setting: since the GP emulator must learn the pricing function reasonably well everywhere in parameter space, and since the region in which real-data calibration will land is not known in advance, evenly-spread samples are considerably more valuable per sample than randomly clustered ones.

4.2 Generating the Synthetic Implied Volatility Surface

After sampling Heston parameter sets, we use each sampled parameter vector to generate a synthetic implied-volatility surface. Each sampled parameter set is of the form

$$\boldsymbol{\theta}_j = (v_{0,j}, \kappa_j, \theta_j, \sigma_j, \rho_j)\,,$$

where j indexes the sampled Heston parameter set. In our notebook, we generate 1,500 such parameter sets using Latin Hypercube Sampling. For each sampled parameter set, we price European call options over a fixed grid of strikes and maturities, using a normalized underlying price, risk-free rate, and dividend yield. Rather than choosing strikes directly, we specify a moneyness grid (strike values relative to the current stock price),

$$\frac{K}{S_0} \in [K_{\min}, K_{\max}],$$

so that the corresponding strikes are

$$K = S_0 \times \text{moneyness}.$$

The maturity grid is chosen in calendar days and then converted into years. Thus, for each Heston parameter set, the notebook prices options across a grid of strikes and maturities, generating multiple synthetic option observations per parameter set. For a given parameter

set $\boldsymbol{\theta}_j$, strike K_i, and maturity T_i, the full synthetic input is

$$\mathbf{x}_{ij} = \begin{pmatrix} v_{0,j} \\ \kappa_j \\ \theta_j \\ \sigma_j \\ \rho_j \\ K_i \\ T_i \end{pmatrix}.$$

The FFT pricer is then used to compute the corresponding Heston call price,

$$C_{\text{Heston}}(\mathbf{x}_{ij}),$$

which is converted into a Black–Scholes implied volatility by solving

$$C_{\text{BS}}\left(S_0, K_i, T_i, r, q, \sigma\right) = C_{\text{Heston}}(\mathbf{x}_{ij})$$

for σ. The resulting solution is saved as the synthetic implied volatility,

$$IV_{\text{synthetic}}(\mathbf{x}_{ij}).$$

Each row of the synthetic dataset therefore contains the sampled Heston parameters, the option contract inputs, the model-generated call price, and the corresponding implied volatility:

$$\left(v_0, \kappa, \theta, \sigma, \rho, K, T, C_{\text{Heston}}, IV_{\text{synthetic}}\right).$$

Rows with invalid prices or failed implied-volatility inversions are removed before the dataset is saved. With 1,500 sampled parameter sets priced across a grid of 13 moneyness values and 11 maturities (up to 143 observations per set), and after filtering out invalid prices and failed inversions, this process yields a final synthetic dataset of approximately 52,000 labeled rows, each pairing a Heston parameter set, strike, and maturity with its corresponding implied volatility.

5 Gaussian Process (GP) Training

5.1 Gaussian Process Emulator

The Carr-Madan FFT pricer can compute Heston option prices efficiently, but repeatedly evaluating it during calibration remains costly. We therefore train a Gaussian Process (GP) emulator to approximate the mapping

$$\mathbf{x} = (v_0, \kappa, \theta, \sigma, \rho, K, T) \longmapsto \sigma_{\text{IV}},$$

where $\mathbf{x}$ contains the five Heston parameters, strike K, and maturity T, and σ_{IV} is the corresponding implied volatility.

A GP models the unknown function $f(\mathbf{x})$ as a distribution over smooth functions,

$$f(\mathbf{x}) \sim \mathcal{GP}\left(0, k(\mathbf{x}, \mathbf{x}')\right),$$

where the kernel $k(\mathbf{x}, \mathbf{x}')$ measures the similarity between two input points. We use a radial basis function (RBF) kernel [5],

$$\exp\left[-\frac{1}{2}\sum_{j=1}^{d}\frac{(x_j - x_j')^2}{\ell_j^2}\right],$$

with a separate length scale ℓ_j for each input variable. Points that are close in the scaled parameter space are assumed to have similar implied volatilities.

For a new input $\mathbf{x}_*$, the trained GP returns a Gaussian predictive distribution,

$$f(\mathbf{x} * *) \mid \mathcal{D} \sim \mathcal{N}\left(\mu * *, \sigma_*^2\right),$$

where μ_* is the predicted implied volatility and σ_* represents the model's predictive uncertainty. Predictions are generally more certain near the training data and less certain in sparsely sampled regions.

The main limitation of an exact GP is its computational cost. Training requires factorizing an $N \times N$ covariance matrix, giving approximately $O(N^3)$ time and $O(N^2)$ memory complexity. For this reason, the exact GP in this project is fitted using a subset of the available training data.

5.2 Sparse Gaussian Process

The variational sparse-GP framework used in this project follows the approach introduced by Titsias [7]. This work developed a principled inducing-point approximation in which the kernel hyperparameters and inducing-point locations are selected by maximizing a variational lower bound to the exact GP log marginal likelihood. The method approximates the posterior of the original GP rather than simply replacing it with a smaller GP, helping to control the information lost through sparsification.

A sparse GP reduces the cost of an exact GP by representing the full training dataset using a smaller set of M inducing points,

$$Z = \mathbf{z}_1, \ldots, \mathbf{z}_M, \qquad M \ll N.$$

Instead of constructing and factorizing the full $N \times N$ covariance matrix, the model works primarily with the covariance matrices between the inducing points and the training data,

$$K_{MM} = k(Z, Z), \qquad K_{MN} = k(Z, X).$$

This reduces the approximate training cost from $O(N^3)$ to $O(NM^2)$. The sparse GP can therefore use all available training rows while retaining a manageable number of inducing points.

In our implementation, the inducing points are placed at the centers obtained from k-means clustering of the scaled training inputs. The model then optimizes a variational evidence lower bound, or ELBO,

$$\mathcal{L}_{\mathrm{ELBO}} \leq \log p(\mathbf{y} \mid X),$$

which provides a tractable approximation to the exact GP marginal likelihood.

The sparse posterior is an approximation and may be less accurate than an exact GP trained on the same full dataset. However, the comparison in this project is also affected by data usage: the exact GP is restricted to 2,500 training points, whereas the sparse GP uses all 41,493 training points summarized by 2,500 inducing points. The sparse method therefore trades an approximate posterior for access to substantially more training information.

5.3 Training and Validation

The exact and sparse GP emulators are trained in the notebooks `train_gp_emulator_sklearn.ipynb` and `train_sparse_gp_emulator_sklearn.ipynb`, respectively. The implementation relies mainly on `scikit-learn` for data splitting, preprocessing, kernel construction, clustering, model fitting, prediction, and evaluation. `NumPy` and `pandas` are used for numerical operations and data handling, while `SciPy` provides the optimization and linear-algebra routines required by the sparse GP implementation. The validation plots are produced with `Matplotlib`.

In both notebooks, the simulated dataset is divided into an 80% training set and a 20% validation set using `train_test_split`. The seven input variables are standardized with `StandardScaler`, which is fitted only on the training data and then applied unchanged to the validation data. This ensures that variables with different numerical scales contribute comparably to the kernel distance.

In the exact GP training, the emulator is constructed using `GaussianProcessRegressor` with an `RBF` kernel and a `WhiteKernel` noise term. Because exact GP training scales cubically with the number of training points, the model is fitted using a reproducible random subset of 2,500 training examples. The `fit` method optimizes the kernel hyperparameters, while `predict(...,return_std=True)` returns both the predicted implied volatility and its predictive standard deviation.

In the SGP notebook, all 41,493 training examples are retained. The training inputs are summarized by 2,500 inducing points selected using `KMeans`. The sparse variational objective is evaluated using the `RBF` kernel together with the `SciPy` functions `cholesky`, `solve_triangular`, and `minimize`. The kernel length scales and noise level are obtained by minimizing the negative ELBO with the L-BFGS-B algorithm.

Both emulators are evaluated on the same 10,374-point validation set using the mean absolute error, root mean squared error, maximum absolute error, and R^2. Figures 10 and 11 compare the predicted and true implied volatilities. The diagonal line represents perfect agreement,

$$\widehat{\sigma}_{\mathrm{IV}} = \sigma_{\mathrm{IV}}.$$

For both models, most points lie close to the diagonal, showing that the emulators recover the implied-volatility mapping accurately over most of the validation domain. The sparse GP remains close to the exact GP despite using an approximate posterior, while also benefiting from the full training dataset. The small number of points farther from the diagonal identifies

regions where the emulator is less accurate and where its uncertainty estimates should be examined more carefully.

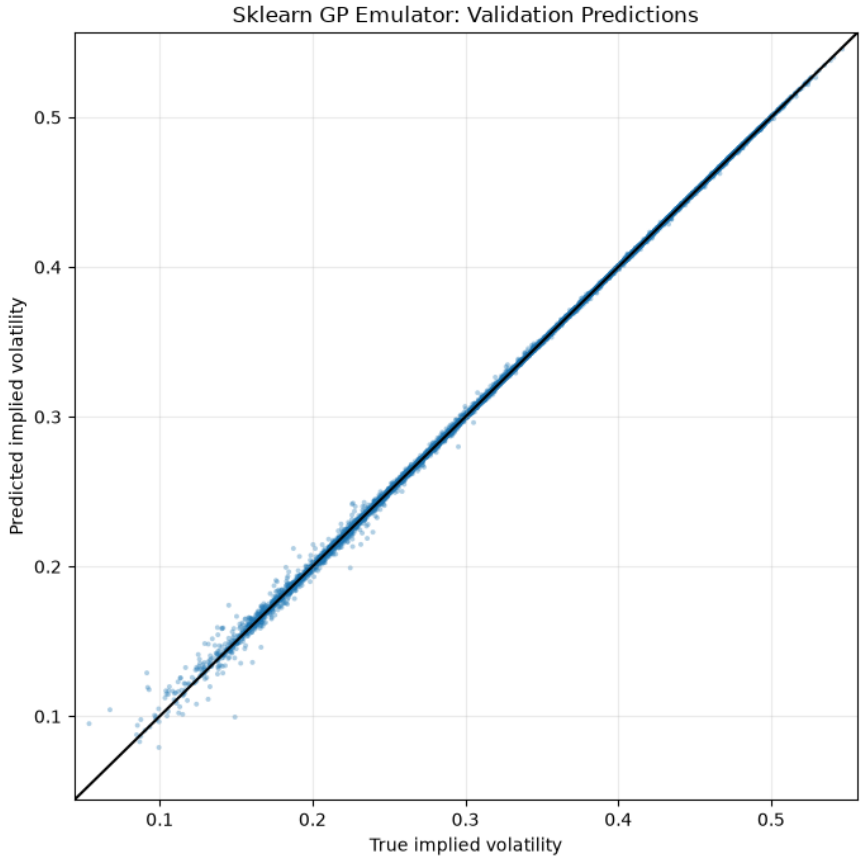

Figure 10: Predicted versus true implied volatility for the exact GP emulator. The model was fitted using 2,500 randomly selected training points and evaluated on 10,374 validation points. The diagonal line represents perfect prediction.

6 Calibration

Having trained the GP and sparse GP emulators on synthetic data (Section 5), we now calibrate the Heston model to real market data. Calibration means finding the set of Heston parameters $(v_0, \kappa, \theta, \sigma, \rho)$ that best reproduces the observed market-implied volatilities for a real option chain, and we carry this out using four different forward models: the direct Carr-Madan FFT pricer, the exact GP emulator, the sparse GP emulator, and Monte Carlo simulation. Section 6.1 describes the real SPY options dataset used for calibration and how it is cleaned; Section 6.2 describes the optimization pipeline shared by all four methods; and Section 6.3 explains how the four `calibrate_*.ipynb` notebooks differ in how they each compute the model-implied volatility inside that pipeline.

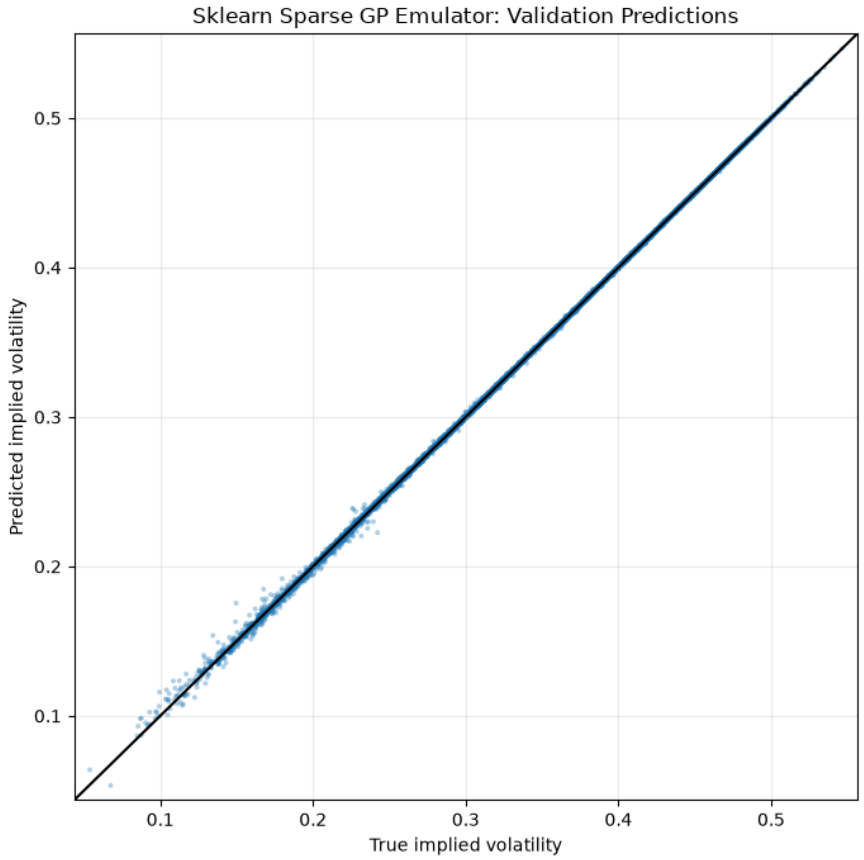

Figure 11: Predicted versus true implied volatility for the sparse GP emulator. The model uses all 41,493 training points, represented by 2,500 inducing points, and is evaluated on 10,374 validation points. The diagonal line represents perfect prediction.

6.1 Market Data

We use a single snapshot of SPY option-market data obtained through the `yfinance` Python package. The collection script downloads SPY price history for the previous two years and the complete call and put chains for the first six available expiration dates. For each contract, the dataset contains the strike, expiration date, option type, bid, ask, trading volume, open interest, and market-implied volatility. We also compute the mid-price as

$$\text{mid}_i = \frac{\text{bid}_i + \text{ask}_i}{2}.$$

Before calibration, we remove quotes that are unlikely to represent reliable market prices. Contracts are excluded sequentially if they have a zero bid, open interest of at most one contract, or a bid-ask spread greater than 30% of the mid-price:

$$\frac{\text{ask}_i - \text{bid}_i}{\text{mid}_i} > 0.30.$$

These filters remove stale, illiquid, and unusually wide quotes that could otherwise place excessive weight on noisy implied-volatility observations. Of the 1,515 raw contracts, 230

are removed because of a zero bid, 109 because of low open interest, and 203 because of a wide relative spread. The final calibration dataset contains 973 observations, split almost evenly between calls and puts. The spot price and dividend yield are obtained from the accompanying SPY price history. The spot price is set to the latest available closing price at the time of data collection, and the annual dividend yield is approximated by dividing the cash dividends paid during the previous 365 days by this spot price:

$$q = \frac{\text{trailing 12-month dividends}}{S_0}.$$

For reproducibility, the calculated spread, relative spread, spot price, and dividend yield are added to every retained row. The cleaned dataset used for calibration is saved as `data/spy_options_clean.csv`.

6.2 Calibration pipeline

After training on synthetic data, we calibrate the emulator against real market data.

1. We begin with an initial guess for the five Heston parameters:

   $$v_0,\ \kappa,\ \theta,\ \sigma,\ \rho.$$

   For each real option contract i, we also know the market-observed strike K_i and maturity T_i. Combining these gives the full input vector to the emulator: $\mathbf{x}_i = (v_0, \kappa, \theta, \sigma, \rho, K_i, T_i)$. The emulator then predicts the corresponding model-implied volatility,

   $$IV_{\text{model}}(\mathbf{x}_i).$$

2. We compare this model-implied volatility to the observed market-implied volatility:

   $$IV_{\text{model}}(\mathbf{x}_i) - IV_{\text{market}}(K_i, T_i).$$

   In code, we use SciPy's `minimize` function to minimize the total squared error across all N option contracts:

   $$\sum_{i=1}^{N} \left[IV_{\text{model}}(\mathbf{x}_i) - IV_{\text{market}}(K_i, T_i)\right]^2.$$

3. The optimizer iterates by adjusting only the five Heston parameters,

   $$v_0,\ \kappa,\ \theta,\ \sigma,\ \rho,$$

   seeking the values that minimize the discrepancy between model and market IVs. Throughout calibration, the real data inputs K_i, T_i, and $IV_{\text{market}}(K_i, T_i)$ remain fixed.

6.3 Differences Between FFT, GP, Sparse GP, and Monte Carlo Calibration

The four `calibrate_*.ipynb` notebooks all solve the same optimization problem. In each case, the optimizer searches over the five Heston parameters

$$(v_0, \kappa, \theta, \sigma, \rho)$$

and minimizes the squared difference between model-implied volatilities and market-implied volatilities:

$$\min_{v_0,\kappa,\theta,\sigma,\rho} \sum_{i=1}^{N} \left[IV_{\text{model}}(\mathbf{x}_i) - IV_{\text{market}}(K_i, T_i)\right]^2 .$$

The difference between the notebooks is not the calibration objective, but how each notebook computes IV_{model}.

In the FFT calibration notebook, IV_{model} is computed using the Carr–Madan FFT Heston pricer. For each candidate parameter set, the notebook prices options directly under the Heston model. These model option prices are then converted into implied volatilities and compared to the market implied volatilities. This is the most direct calibration method because it evaluates the Heston pricing model itself during each optimizer step. However, it is also the most computationally expensive of the semi-analytic methods, since the FFT pricer must be called repeatedly throughout the minimization.

In the GP calibration notebook, the direct FFT pricing step is replaced by a trained Gaussian Process emulator. The emulator takes the input $\mathbf{x}_i = (v_0, \kappa, \theta, \sigma, \rho, K_i, T_i)$ and directly predicts the implied volatility: $IV_{\text{model}}(\mathbf{x}_i) = IV_{\text{GP}}(\mathbf{x}_i)$. Thus, the GP notebook does not compute option prices during calibration. Instead, it uses the emulator as a surrogate for the Heston implied-volatility surface. This makes calibration faster than the direct FFT approach, but the result depends on how accurately the GP emulator learned the synthetic Heston training data.

In the sparse GP calibration notebook, the calibration objective is again unchanged, but the emulator is a sparse Gaussian Process rather than a full Gaussian Process. The sparse GP uses a smaller set of representative inducing points instead of the full synthetic training set. Therefore,

$$IV_{\text{model}}(\mathbf{x}_i) = IV_{\text{sparse GP}}(\mathbf{x}_i).$$

This further reduces the computational cost of evaluating the emulator during calibration. The tradeoff is that the sparse GP is an additional approximation: it approximates the full GP, which itself approximates the Heston implied-volatility surface.

In the Monte Carlo calibration notebook, IV_{model} is computed by simulating Heston stock-price paths under each candidate parameter set and pricing the option as the discounted expectation of its payoff over these simulated paths. As with the FFT notebook, this requires no emulator, but replaces the semi-analytic FFT pricer with direct simulation:

$$IV_{\text{model}}(\mathbf{x}_i) = IV_{\text{MC}}(\mathbf{x}_i).$$

This is the most computationally expensive of the four methods, since a full set of stock-price paths must be simulated and repriced at every optimizer step, but it makes no approximation

to the option-pricing model itself beyond simulation (Monte Carlo) error.
Therefore, the four notebooks differ only in the forward model used inside the optimizer:

$$IV_{\text{model}} = \begin{cases} IV_{\text{FFT}}, & \text{direct Heston FFT calibration,} \\ IV_{\text{GP}}, & \text{full Gaussian Process emulator calibration,} \\ IV_{\text{sparse GP}}, & \text{sparse Gaussian Process emulator calibration,} \\ IV_{\text{MC}}, & \text{Monte Carlo simulation calibration.} \end{cases}$$

7 Results and Discussion

In this section, we compare the outputs of the calibrations. We evaluate the FFT, GP, sparse GP, and Monte Carlo calibrations along four axes: the fitted Heston parameters themselves, the resulting fit to the market volatility smile, calibration runtime, and for the two GP-based methods, we analyze the reliability of their predictive uncertainty. Our comparison analysis is in notebook `compare_calibrations_4.ipynb`, and our uncertainty analysis in `gp_uncertainty_analysis.ipynb`.

7.1 Parameter Comparison

Figure 12 shows the Heston parameters obtained after calibrating each of the four methods to the SPY dataset described in Section 6.1. Although there is difference between the parameter values calibrated with different methods, they yield consistent IV, as shown in Figure 13.

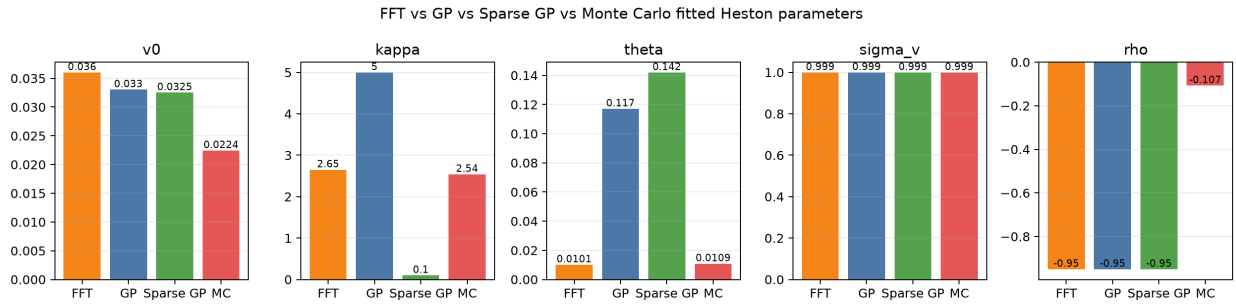

Figure 12: Heston parameter values obtained after calibration to SPY data. Different color assignments indicate different methods used in calibration.

The four methods broadly agree on σ, which calibrates close to its upper bound of 0.999 in all cases, and on v_0, which stays within a narrow band across methods. The largest disagreement is in κ and θ: the sparse GP calibrates to a much smaller mean-reversion rate than the other three methods, while the GP calibrates to the largest; on θ, FFT and Monte Carlo agree closely near 0.01, while GP and sparse GP both calibrate to a substantially higher value near 0.12–0.14. Only Monte Carlo recovers a ρ noticeably different from the strongly negative values found by the other three methods. Since κ and θ trade off against one another in their effect on the variance path (Section 2.2), and since the optimizer only observes their combined effect on the implied-volatility surface, some degree of non-identifiability between

these parameters across methods is expected; the more informative comparison is therefore the resulting fit quality, discussed next.

7.2 Fit Quality: Market Smile Comparison

After calibration, we evaluated each method by predicting implied volatilities across a range of strike prices for two different expiration dates and compared the predictions with the observed market implied volatilities. The comparison is shown in Figure 13.

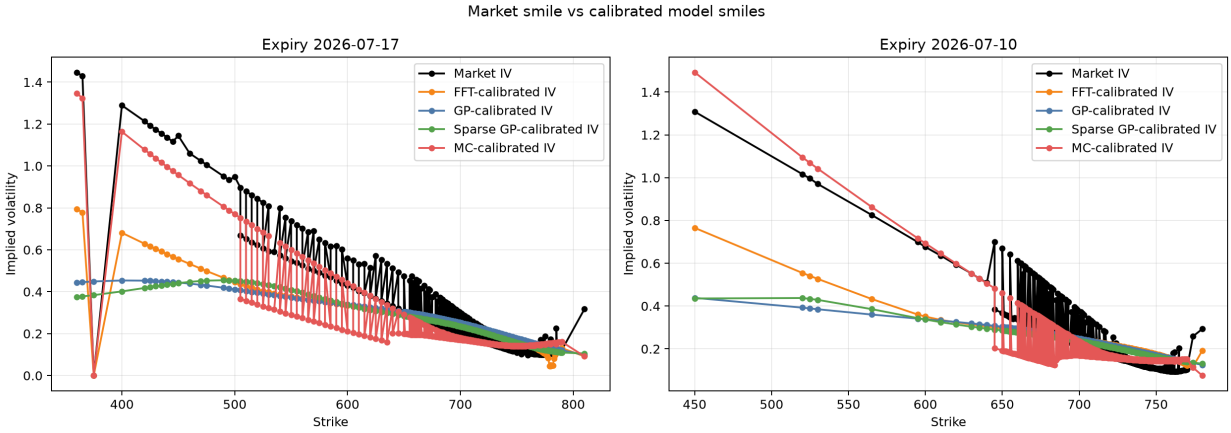

Figure 13: Real market IV compared with the IVs obtained with different calibration methods to the SPY stock at two different maturity dates. We see that MC calibration yields a better approximation and can reproduce the sharp changes in IV. On the other hand, GP, sparse-GP, and FFT calibrated IVs tend to follow a linear behavior in mapping.

The Monte Carlo calibration is the only method that reproduces the sharp, non-monotonic changes in the market smile at both expiries; the FFT, GP, and sparse GP calibrations instead settle on IV curves that vary roughly linearly with strike, systematically under- and over-shooting the market smile in different strike regions. This is consistent with the parameter comparison in Figure 12: because the GP and sparse GP emulators were trained on a synthetic surface generated from a bounded moneyness and maturity grid (Section 4.2), their predictions are smoothed by the RBF kernel's length scales and cannot reproduce local curvature in the market smile as sharply as a method that reprices the Heston model directly (FFT) or by simulation (Monte Carlo). The gap between Monte Carlo and the other three methods therefore reflects the added approximation layer each method introduces: FFT approximates the pricing integral semi-analytically, while GP and sparse GP additionally approximate the FFT-generated training surface with a smooth kernel regression.

A caveat applies specifically to the Monte Carlo result. Unlike FFT, GP, and sparse GP which are deterministic given a parameter set, the Monte Carlo pricer introduces its own simulation noise into every evaluated price, and this noise was held fixed (via a fixed random seed) throughout calibration. It is therefore possible that some portion of Monte Carlo's apparent ability to track the sharp, non-monotonic features of the market smile reflects the optimizer fitting one noisy curve (simulation noise) to another (market microstructure noise from the short-dated, thinly-traded contracts described in Section 6.1), rather than the

Heston model under Monte Carlo pricing genuinely capturing the market's local curvature. Distinguishing these two explanations would require extended work re-evaluating the Monte Carlo-calibrated parameters with an independent random seed and checking whether the fit quality persists.

7.3 Efficiency: Calibration Error vs. Runtime

Figure 14 plots calibration error (RMSE) against calibration time for all four methods, combining runtime and fit quality into a single view. Sparse GP was the fastest at 3.26 s, followed closely by FFT (3.50 s) and GP (3.88 s); Monte Carlo was by far the slowest at 16.46 s, roughly 5× the sparse GP time. These times include the full optimizer run, not a single model evaluation, so they reflect the complete calibration workflow described in Section 6.2.

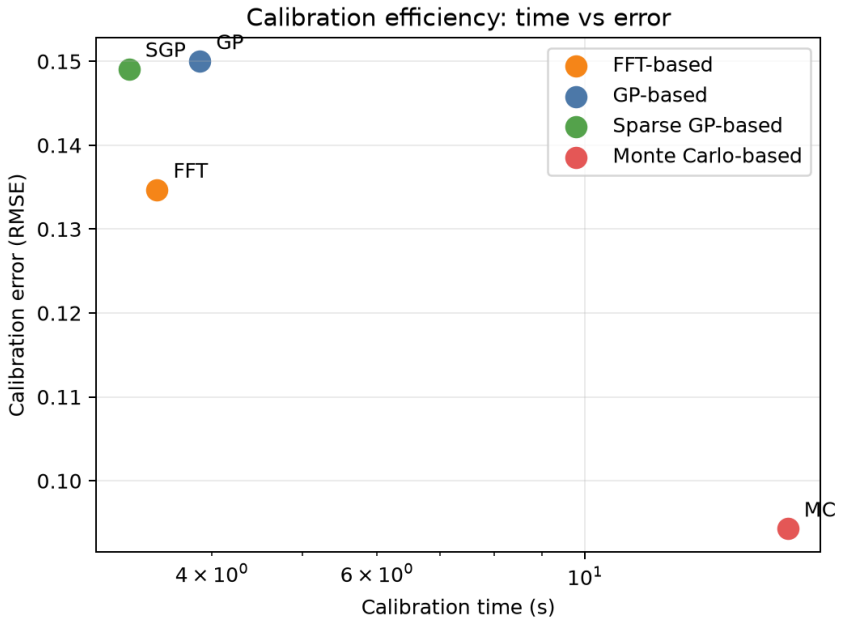

Figure 14: Heston parameter calibration error vs calibration time. It can be seen that although FFT, GP and SGP calibrate relatively quickly, they yield a larger error in comparison to MC methods.

This plot makes the central trade-off of the project explicit: FFT, GP, and sparse GP occupy the fast, higher-error region (RMSE $\approx$ 0.135–0.150, all under 4 s), while Monte Carlo occupies the slow, lower-error region (RMSE 0.094 in 16.46 s). Consistent with the motivation in Section 1, the GP-based methods do not show a decisive runtime advantage over direct FFT calibration here; because the option chain used for calibration is relatively small (973 contracts), the per-call cost of the FFT pricer is not yet large enough for emulator substitution to pay off. The benefit of the emulator approach would be expected to grow with the size of the option chain or the number of optimizer iterations required to converge.

A natural extension of this project would therefore be to scale up the calibration workload itself. For example, by calibrating against a much larger option chain (more strikes,

maturities, or multiple underlyings simultaneously), or by using an optimizer that requires substantially more forward-model evaluations to converge. Under either change, the fixed per-call cost of the FFT pricer would be paid far more often, while the emulator's per-call cost would remain roughly constant; this is the regime in which the GP and sparse GP methods would be expected to pull ahead of direct FFT calibration in wall-clock time. Separately, training the emulators on a denser or wider-ranging synthetic dataset may also narrow the fit-quality gap to Monte Carlo seen in Figure 13, since some of the smile-smoothing behavior observed here may reflect the coarseness of the synthetic training grid rather than a fundamental limitation of the GP approach.

7.4 Predictive Uncertainty: GP vs. Sparse GP

A key advantage of a GP emulator over a plain point predictor is that every prediction comes with a predictive standard deviation that quantifies how confident the model is. We examine this uncertainty for both emulators along two axes: whether the reported uncertainties are calibrated on held-out data (the validation set from Section 5.3, which the emulators never saw during training), and whether the uncertainty grows appropriately under extrapolation. In short, we want to know whether σ_* can be trusted: does it match the model's true error rate, and does it correctly grow when the model is guessing?

7.4.1 How each model computes uncertainty

Beyond a single predicted number, a GP emulator can also tell us how much to trust that prediction. Intuitively, a GP works by comparing a new query point $\mathbf{x}_*$ (a candidate set of Heston parameters, strike, and maturity) to all the points it saw during training. If $\mathbf{x}_*$ is similar to many training points measured by the kernel $k(\cdot, \cdot)$ from Section 5.1, which is large when two inputs are close and small when they are far apart, the model has seen this kind of input before and reports a small uncertainty. If $\mathbf{x}_*$ is far from anything in the training set, the model has little to go on and reports a large uncertainty instead. This is what distinguishes a GP from a plain point predictor: it does not just give an answer, it also provides a distribution.

Concretely, both emulators return, for an input $\mathbf{x}_*$, a predicted implied volatility μ_* (the posterior mean) together with a standard deviation σ_* that measures this uncertainty, reported in IV units. A small σ_* means the prediction is well-supported by nearby training data; a large σ_* means the model is extrapolating. The formulas below [6] make this precise for each emulator, but the underlying idea is the same in both cases: σ_* starts at the model's total prior uncertainty, $k(\mathbf{x}_*, \mathbf{x}_*)$, and is reduced by however much the training data "explains" the query point.

For the exact GP, with training inputs X and $K = k(X, X) + \sigma_n^2 I = LL^\top$,

$$\sigma_*^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \left\| L^{-1} k(X, \mathbf{x}_*) \right\|^2.$$

The subtracted term grows when $\mathbf{x}_*$ is close to training points (making $k(X, \mathbf{x}_*)$ large), pulling σ_*^2 toward zero; far from the data this term shrinks, and σ_*^2 rises back toward the prior variance $k(\mathbf{x}_*, \mathbf{x}_*)$.

For the sparse GP, the same idea holds, but the comparison is made against the M inducing points Z (Section 5.2) rather than every training row. With $K_{mm} = LL^\top$, $B = I + \sigma_n^{-2} L^{-1} K_{mn} K_{nm} L^{-\top} = L_B L_B^\top$, and $w = L^{-1} k(Z, \mathbf{x}_*)$, $v = L_B^{-1} w$,

$$\sigma_*^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \|w\|^2 + \|v\|^2.$$

In practice, the exact GP's saved model already includes the Cholesky factor L needed for this calculation; for the sparse GP, the covariance factors L and L_B are rebuilt from the saved inducing points, length scales, and noise level together with the training inputs.

7.4.2 Are the uncertainties calibrated?

If a model's uncertainty is honest, the true value should fall within $\pm 1\sigma$ of the predicted mean about 68% of the time, and within $\pm 2\sigma$ about 95% of the time. On the held-out validation set, the exact GP's true values fall within 1σ only 60.0% of the time (and within 2σ 81.0% of the time), while the sparse GP does slightly better, at 67.6% and 84.8%. Both emulators are therefore a bit overconfident. Their uncertainty bands are narrower than they should be, but the sparse GP is closer to the ideal.

Figure 15 shows this visually. The left panel shows how large the reported uncertainties typically are: the sparse GP tends to report slightly larger, more spread-out uncertainties than the exact GP, matching the numbers above. The right panel checks something more practical: does a prediction the model flags as "uncertain" actually tend to be wrong? We group predictions into buckets by their reported uncertainty and plot each bucket's average error. For both emulators, the trend rises clearly from left to right. Predictions with higher reported uncertainty do have larger errors on average, confirming that the uncertainty estimates are a useful, if imperfect, warning signal.

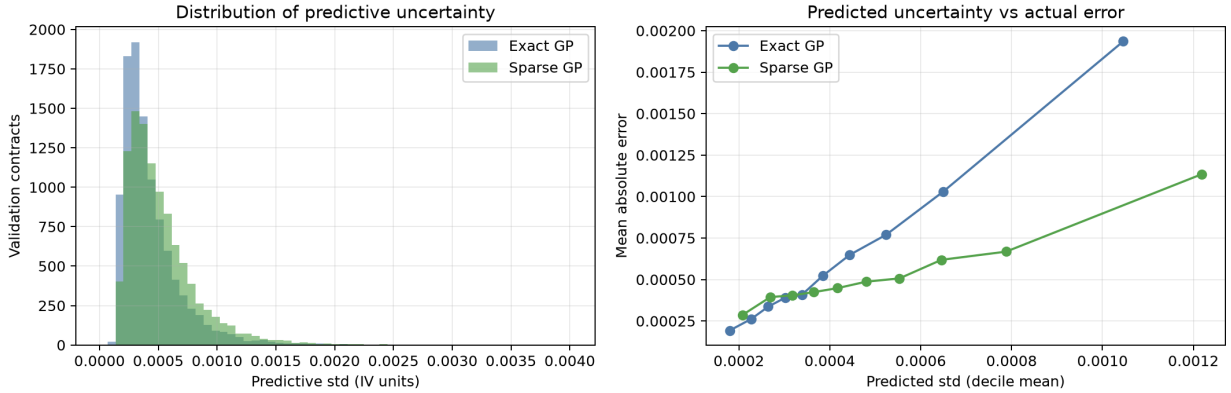

Figure 15: Left: distribution of predictive standard deviations on the validation set for the exact and sparse GP emulators. Right: mean absolute error versus predicted standard deviation, grouped into deciles of predicted std, showing that higher reported uncertainty corresponds to higher realized error for both emulators.

7.4.3 Uncertainty under extrapolation

The most useful property of GP uncertainty is that it increases where there is little or no training data. To test this, we fix the five Heston parameters at their training medians and vary only the strike K, stepping it through a range of values that starts inside the range the emulator was trained on and continues past it into strikes the model never saw during training. At each strike value we record the predicted IV and its uncertainty band, producing a curve that shows how the prediction and its $\pm 2\sigma$ band change as we move from familiar territory into unfamiliar territory (Figure 16).

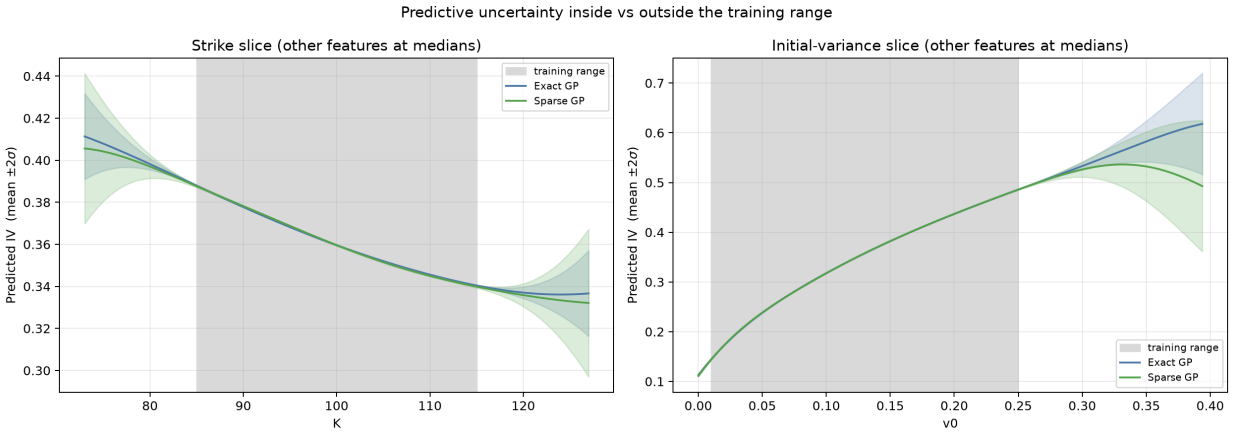

Figure 16: Predicted implied volatility with $\pm 2\sigma$ bands as strike is swept outside the training range, for the exact GP and sparse GP emulators. The shaded region marks the training range.

Averaged over the strike slice, the exact GP's predictive standard deviation widens from 0.00014 inside the training range to 0.00310 outside it, which is a 22.0× increase. While the sparse GP widens from 0.00018 to 0.00573, a 31.0× increase. Both emulators therefore correctly flag out-of-range queries as unreliable, with the sparse GP widening its band somewhat more aggressively than the exact GP.

7.4.4 Discussion

Uncertainty is the GP's headline feature. Both emulators return not just a predicted IV but a standard deviation, and that number is informative: it is small where training data is dense and grows, often by a large multiple, once a query leaves the training range. A point-estimate emulator, such as a neural network trained with plain mean-squared error, gives no comparable trust signal, so a user cannot distinguish an interpolation from a wild extrapolation.

The two models agree closely inside the training data, but their uncertainties differ in interpretable ways. The sparse GP typically reports a slightly larger in-sample standard deviation, since it summarizes the data through M inducing points rather than every row, and this extra posterior variance is the honest price of the approximation. Between and beyond the inducing points, the sparse GP's variance also reverts toward the prior somewhat faster, so its extrapolation bands open up at least as aggressively as the exact GP's.

These bands are latent-function uncertainty, excluding the observation-noise term σ_n^2, and assume the fitted kernel and hyperparameters are correct; genuine model error from an inadequate kernel, such as the smoothing behavior seen in the smile comparison of Section 7.2, is not captured by σ_*. The calibration coverage reported in Section 7.4.2 should therefore be read as a check on the fitted model, not a guarantee. Still, as a relative, monotonic signal of where the emulator is least sure, both GPs deliver exactly the information a point predictor cannot, which is the property motivating their use in the first place.

References

[1] Peter Carr and Dilip Madan. "Option valuation using the fast Fourier transform". In: *Journal of computational finance* 2.4 (1999), pp. 61–73.

[2] John C Cox, Jonathan E Ingersoll, Stephen A Ross, et al. "A theory of the term structure of interest rates". In: *Econometrica* 53.2 (1985), pp. 385–407.

[3] Steven L Heston. "A closed-form solution for options with stochastic volatility with applications to bond and currency options". In: *The review of financial studies* 6.2 (1993), pp. 327–343.

[4] Wilson Tsakane Mongwe, Rendani Mbuvha, and Tshilidzi Marwala. *Bayesian Machine Learning in Quantitative Finance: Theory and Practical Applications*. Springer, 2025.

[5] Pedregosa, F. and Varoquaux, G. and Gramfort, A. and others. *RBF — scikit-learn documentation*. `https://scikit-learn.org/stable/modules/generated/sklearn.gaussian_process.kernels.RBF.html`. Accessed: 2026-07-10. 2026.

[6] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[7] Michalis K. Titsias. "Variational learning of inducing variables in sparse Gaussian processes". In: *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics*. PMLR. 2009, pp. 567–574.